

S C S 1 3 0 3

Introduction to Software Engineering

Section 3 · Agile Software Development

Prof. K. P. Hewagamage

B.Sc. Computer Science · Year 1, Semester 1

University of Colombo School of Computing

Section 3 — Intended Learning Outcomes

By the end of this section, students should be able to:

- LO1 Explain the Agile Manifesto and its 12 principles in plain language
- LO2 Describe Scrum — roles, events, artifacts, and the sprint cycle
- LO3 Explain XP's key practices — TDD, pair programming, and small releases
- LO4 Describe how a Kanban board works and what WIP limits achieve
- LO5 Explain Lean's origins and its seven principles
- LO6 Choose an appropriate agile approach for a given project context
- LO7 Identify the main agile tools (Jira, Trello) and how they support teams
- LO8 Describe how AI is beginning to change agile software development practices

Section 3 — Roadmap

Six topics across 4 lecture hours — building from foundations to modern practice

3.1

Agile culture — rapid recap

3.2

Scrum — the dominant framework

3.3

Extreme Programming (XP)

3.4

Kanban — visualise and flow

3.5

Lean and other agile methods

3.6

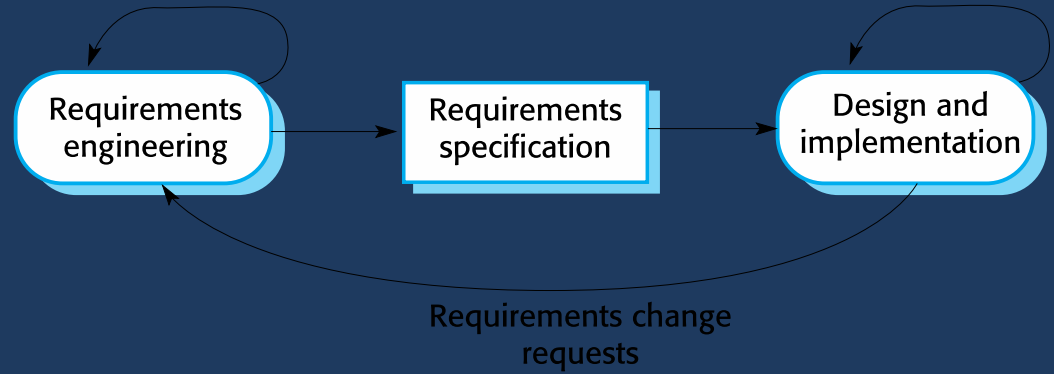
Tools and AI in Agile

3.1

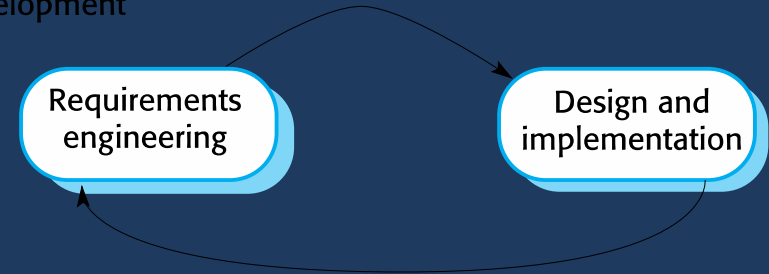
Agile Culture

From plan-driven to AgileHeading towards

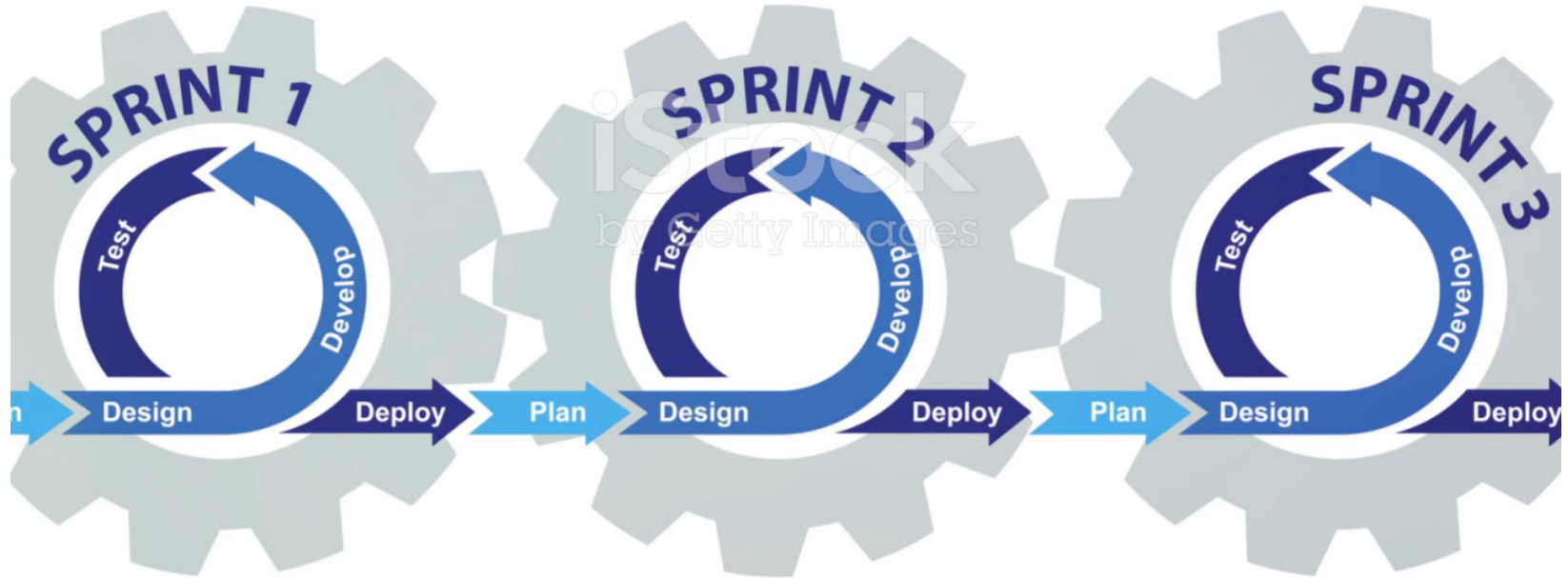
Plan-based development



Agile development



AGILE METHODOLOGY



Why Agile? The Case for Change

Key insight

Plan-driven approaches assume you can fully define requirements upfront. In most modern software projects, **that assumption is wrong.**

Requirements change constantly

Business needs shift, markets evolve, technology advances — *plans made in January are wrong by April*

Customers can't specify upfront

Users don't know exactly what they want until they can see and touch a working system

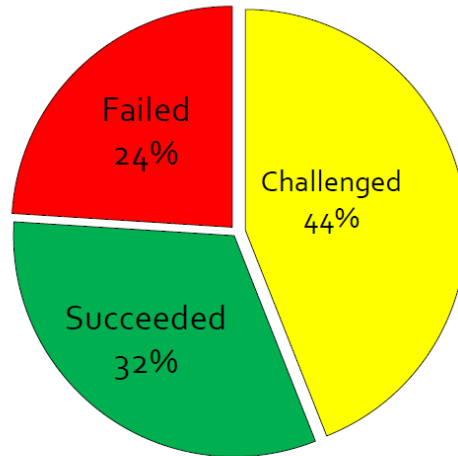
Speed to market is critical

In a competitive environment, releasing working software in weeks beats planning for a year

Teams need flexibility

Rigid processes create bottlenecks; empowered, self-organising teams respond faster

Status of Software Projects in 10 Years ago...

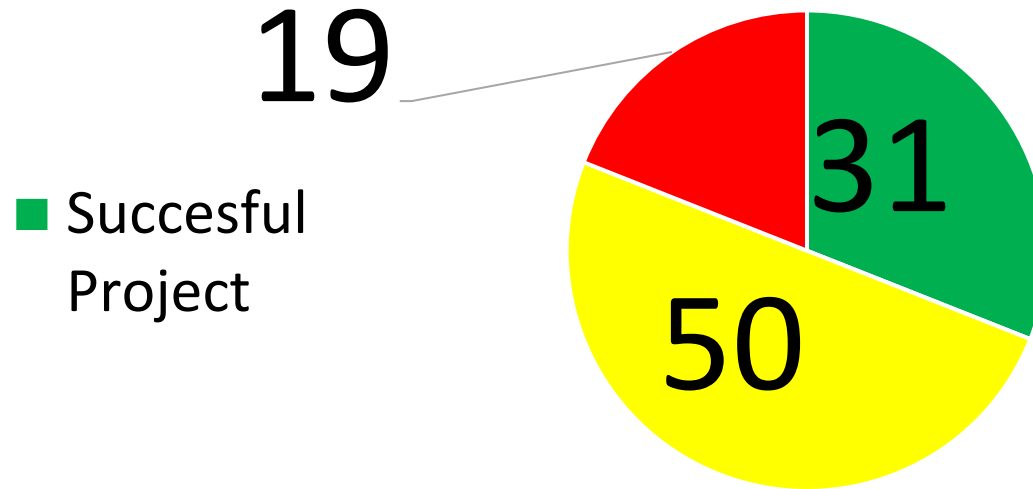


- 32% of all projects succeeded (on time, on budget, with required features)
- 44% are challenged (late, over budget and/or with less than the required features)
- 24% have failed (cancelled prior to completion and/or never used)

Source: Standish CHAOS Report 2009
Boston, Massachusetts, April 23, 2009

Status of Software Projects after 10 Years

Sandish CHOS Report 2024



The Agile Manifesto — Four Values

Snowbird, Utah, 2001 · 17 practitioners · one shared statement

The Manifesto does not say the items on the right have no value. It says we value the items on the left MORE.

Individuals and interactions

over

processes and tools

Working software

over

comprehensive documentation

Customer collaboration

over

contract negotiation

Responding to change

over

following a plan

The Agile Manifesto — 12 Principles (1 of 2)

- | | | |
|---|------------------------------------|--|
| 1 | Satisfy the customer | Highest priority is early and continuous delivery of valuable software |
| 2 | Welcome change | Even late in development — use it for the customer's competitive advantage |
| 3 | Deliver frequently | Working software every few weeks; the shorter the timescale, the better |
| 4 | Work together daily | Business people and developers must collaborate throughout the project |
| 5 | Trust motivated individuals | Give them environment and support; trust them to get the job done |
| 6 | Face-to-face conversation | The most effective method of conveying information within a team |

The Agile Manifesto — 12 Principles (2 of 2)

7

Working software = progress

Working software is the primary measure of progress — not documents

8

Sustainable pace

Sponsors, developers, users should maintain a constant pace indefinitely

9

Technical excellence

Continuous attention to good design and engineering enhances agility

10

Simplicity

Maximise the amount of work NOT done — do only what is needed

11

Self-organising teams

The best architectures and designs emerge from self-organising teams

12

Reflect and adapt

Regularly reflect on how to be more effective and adjust accordingly

The Agile Methods Family

All are built on the Manifesto values — each emphasises something different

Scrum

Framework · Roles ·
Sprints

XP

Engineering excellence

Kanban

Visualise · Flow · WIP
limits

Lean

Eliminate waste · Deliver
fast

Crystal

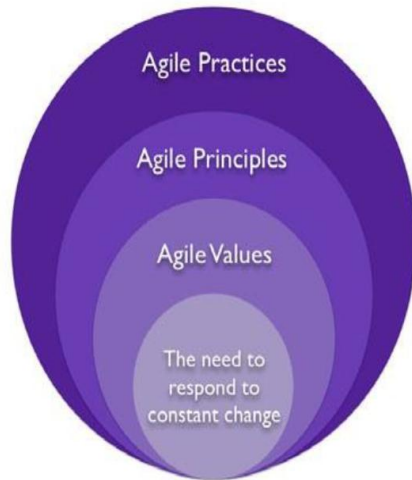
People · Communication ·
Safety

FDD

Feature-by-feature
delivery

We cover Scrum, XP, Kanban, and Lean in depth. Crystal and FDD at overview level.

Relationship between Agile Values, Principles and Practices



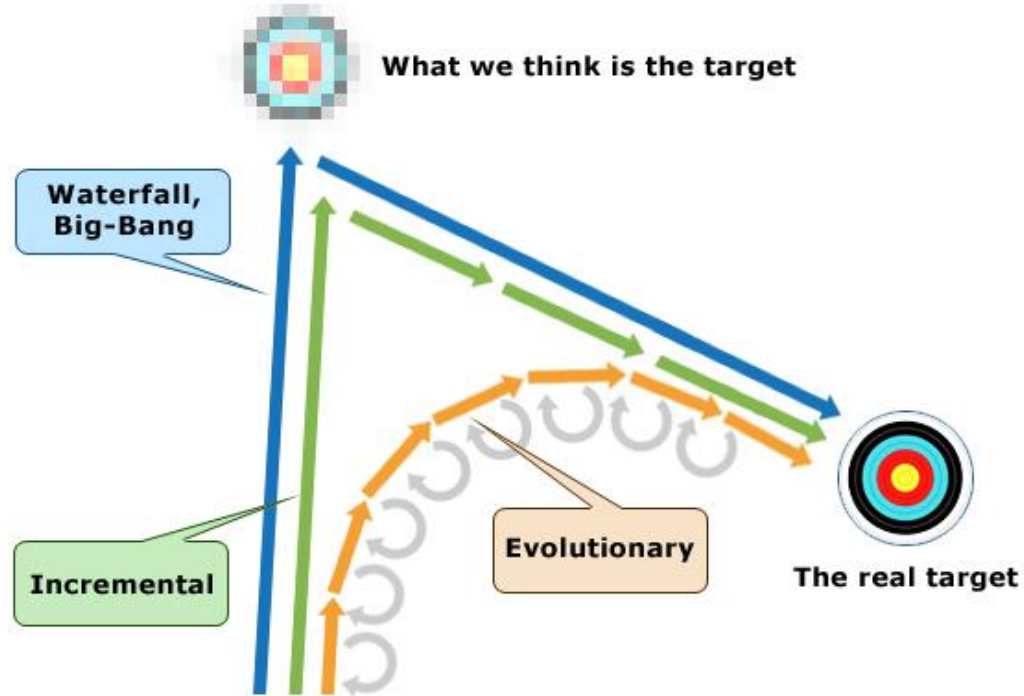
- 4 Values
- 12 Principles
- Many practices, grouped with as methodologies
 - E.g. XP, SCRUM,

Agile Software Development:

Learn → Adapt → Deliver

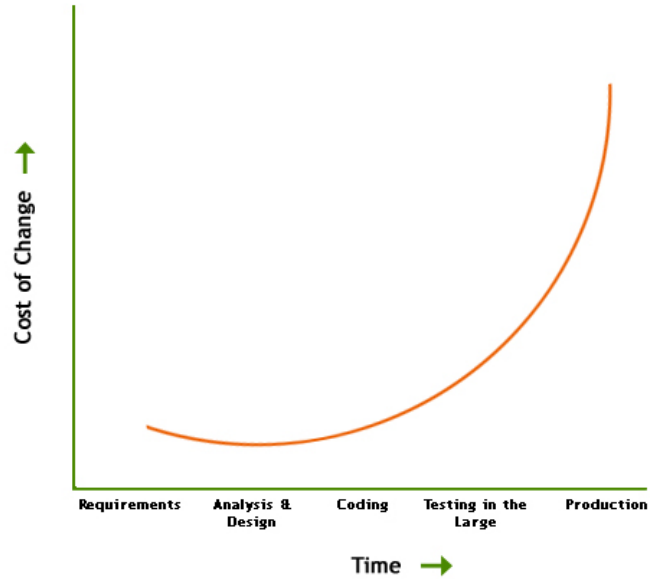
- Ability to adapt
 - To changing requirements, To changing circumstances
- Agile Software development?
 - based on Learn & Adapt rather than rigid processes
 - Evolving systems in short iterations
 - Each release is a working system
 - Focus on business value
 - Design for change
 - Communicating efficiently & real time visibility to progress
 - Leveraging human strengths
 - Engage, align, and empower the team
 - Get power from each member
- Agile is not an excuse for “no discipline”

Reaching the Goal

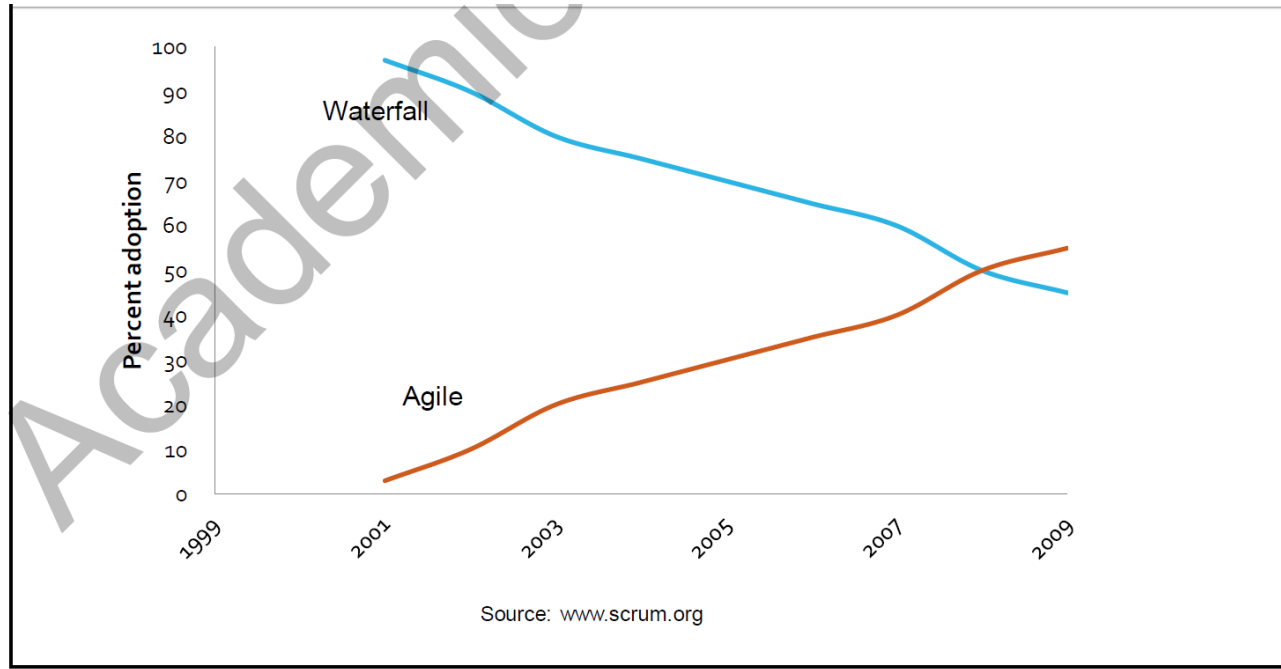


Why Agile?

COST OF CHANGE - WATERFALL



No. of Projects Success



Agile vs. Waterfall Development Success Rates

Agile



Waterfall



Source: The Standish Group

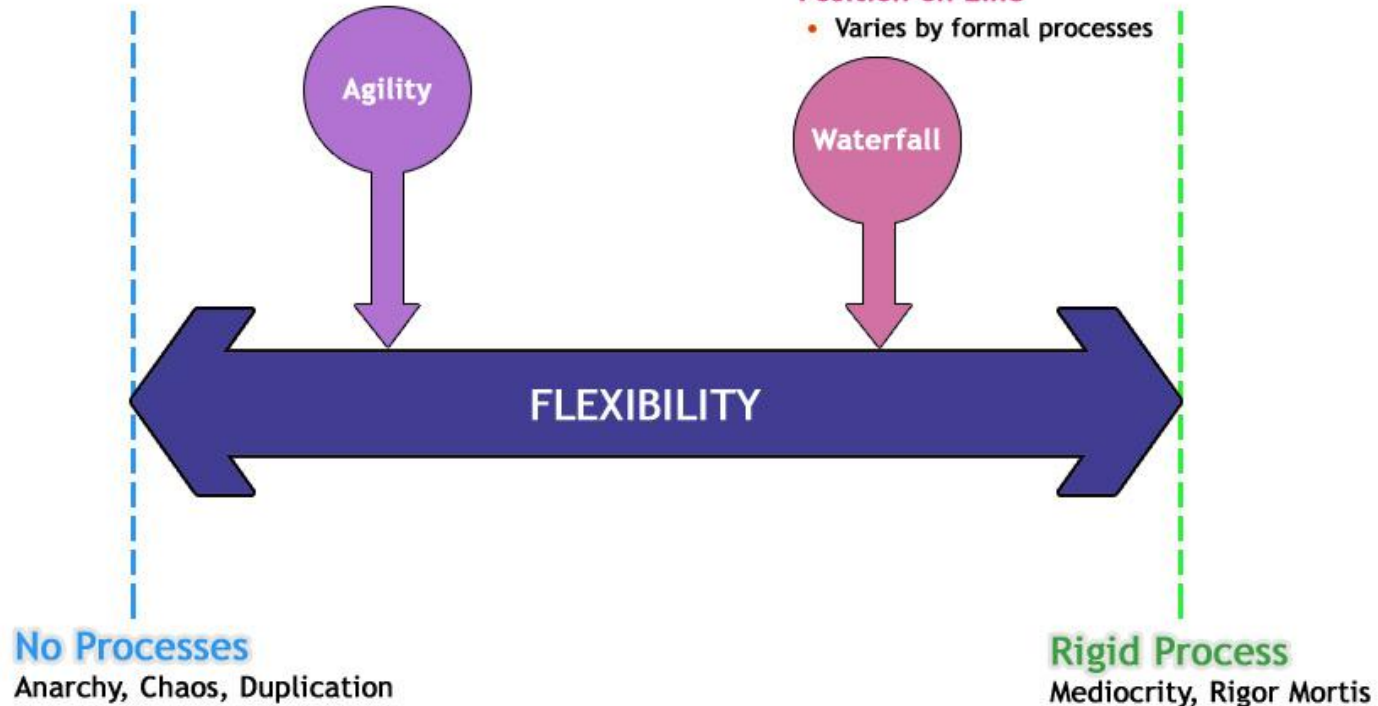
Adaptability Scale

Position on Line

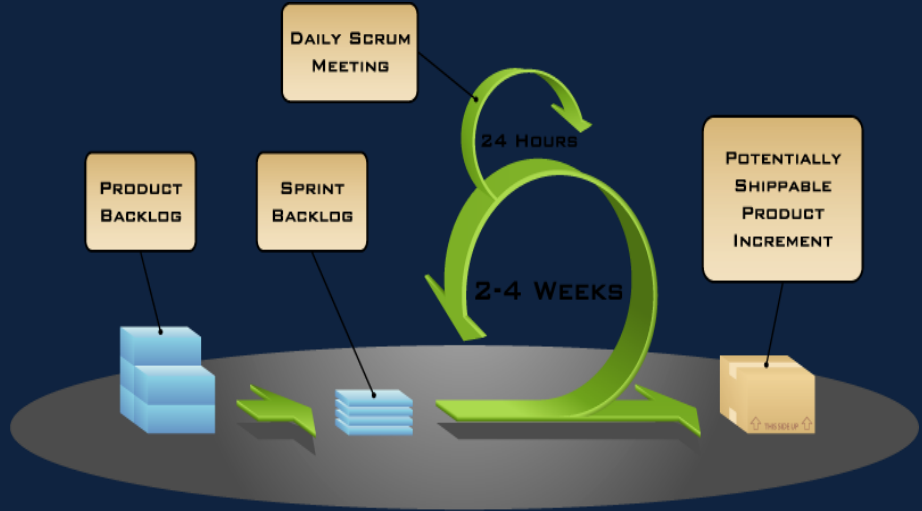
- Varies by Project
- Varies by Risk
- Varies by Personnel

Position on Line

- Varies by formal processes



Scrum



The world's most widely used agile framework —
60%+ market share

What is Scrum?

Definition

Scrum is a lightweight framework that helps teams deliver valuable products through short, repeatable cycles called sprints. It is not a process or a technique — it is a framework within which teams can use various processes and techniques.

Three things that make Scrum distinctive:



Time-boxed sprints

All work happens inside fixed 1–4 week cycles called sprints. The sprint end date never moves.



Inspect and adapt

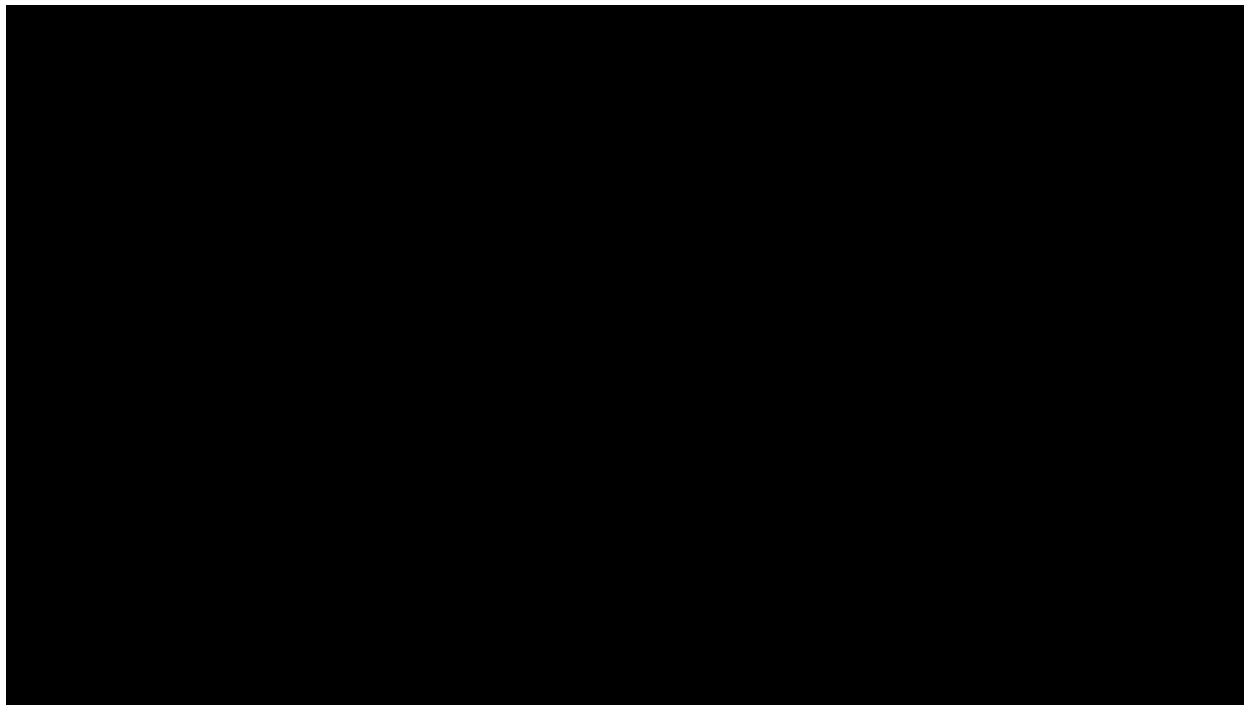
At every sprint boundary, the team examines what they built and improves how they work.



Empirical process control

Decisions are based on observation and experimentation — not a plan made months ago.

Scrum in 10 Minutes



Scrum — Five Core Values

The values are not aspirational — they are the foundation that makes the framework work

Commitment

Every team member commits personally to achieving the sprint goal

Focus

Everyone focuses on the sprint work — interruptions are minimised

Openness

The team and stakeholders are open about all work and all challenges

Respect

Team members treat each other as capable, independent professionals

Courage

Team members have the courage to do the right thing, even when hard

Scrum — Three Roles

These are not job titles — they are accountabilities within the Scrum team

Product Owner

Accountable for: What to build and in what order

- Manages and prioritises the Product Backlog
- Represents the needs of stakeholders and customers
- Accountable for maximising the value of the product
- Makes decisions about scope, priority, and release

Scrum Master

Accountable for: How the team works and improves

- Ensures Scrum is understood and followed correctly
- Facilitates all Scrum events at the right time
- Removes impediments blocking the team's progress
- Is a coach — not a project manager or line manager

Development Team

Accountable for: How to build the increment

- Cross-functional — has all skills to deliver a Sprint
- Self-organising — no one tells them how to do the work
- 3–9 members (optimal for coordination and speed)
- Jointly responsible for delivering the Sprint Goal

Scrum — Three Artifacts

Artifacts make work and progress transparent to everyone

Product Backlog

The ordered list of everything that might be needed in the product. The Product Owner manages it. It is never complete — it evolves as the product and market change.

Examples: user stories, bug fixes, features, improvements — all expressed as value to the customer

Sprint Backlog

The set of Product Backlog items selected for the current Sprint, plus the plan for delivering them and achieving the Sprint Goal. Owned by the Development Team.

The team creates it at Sprint Planning and updates it daily. Only the team can change it during the Sprint.

Product Increment

The sum of all completed Product Backlog items at the end of a Sprint. Must meet the Definition of Done and be potentially shippable — even if the team decides not to release.

Each Sprint adds to the increment. After 6 sprints, the increment includes everything built in all 6 sprints.

Scrum — Five Events

Every event has a maximum time-box — it ends when its purpose is achieved

1

Sprint

1–4 weeks

The container for all other events. A time-box within which the team delivers a potentially shippable increment.

2

Sprint Planning

Max 8 hours

The team collaborates to define the Sprint Goal and select the backlog items they will deliver in this Sprint.

3

Daily Scrum

15 minutes

A daily synchronisation meeting. Three questions: What did I do yesterday? What will I do today? Any blockers?

4

Sprint Review

Max 4 hours

The team demonstrates the increment to stakeholders and collects feedback to update the Product Backlog.

5

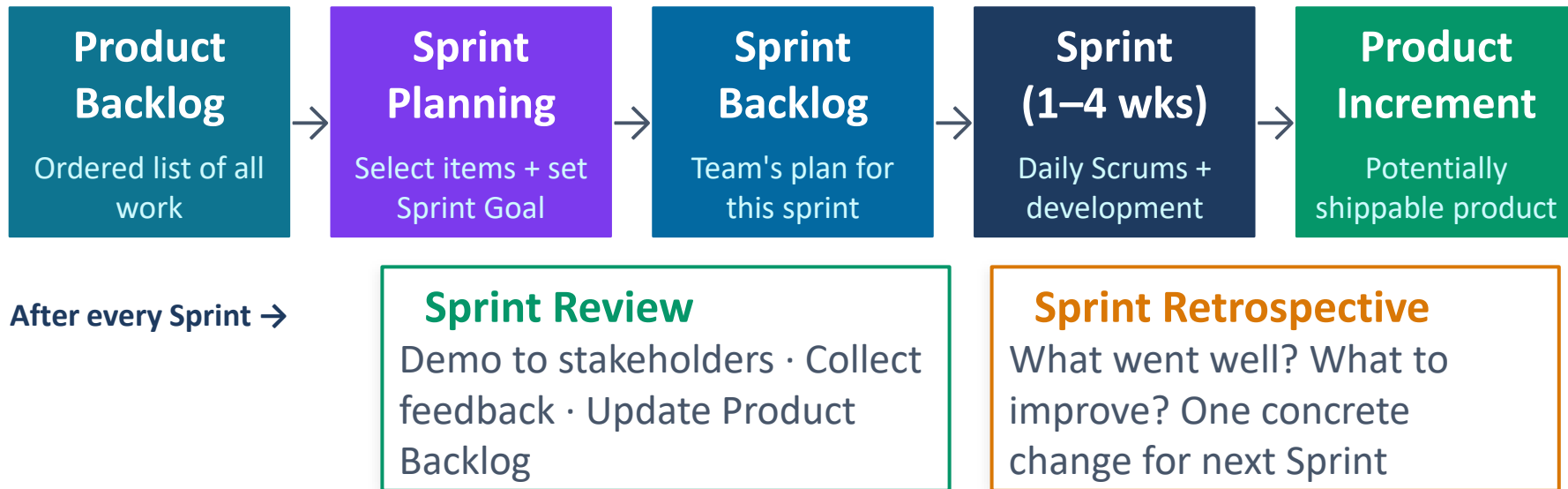
Sprint Retrospective

Max 3 hours

The team reflects on how they worked and identifies one or two concrete improvements for the next Sprint.

Scrum — The Sprint Cycle

Every sprint follows the same repeating pattern — inspect, adapt, deliver, repeat



*The output of Sprint Review feeds back into the Product Backlog
→ ready for the next Sprint Planning.*

Scrum — Definition of Done (DoD)

One of Scrum's most important quality concepts

What is it?

The Definition of Done is a shared understanding of what "complete" means. A Product Backlog item is not done until it meets every criterion in the DoD.

Why the Definition of Done matters:

- It creates transparency — everyone knows exactly what 'done' looks like
- It prevents undone work from accumulating as hidden technical debt
- Without it, different team members have different standards — leading to surprises at Sprint Review
- The DoD evolves over time as the team improves — stricter DoD = higher quality

*Example DoD: code written · unit tests pass · code reviewed · integrated into main ·
deployed to test environment · acceptance criteria verified*

Scrum — Strengths and Challenges

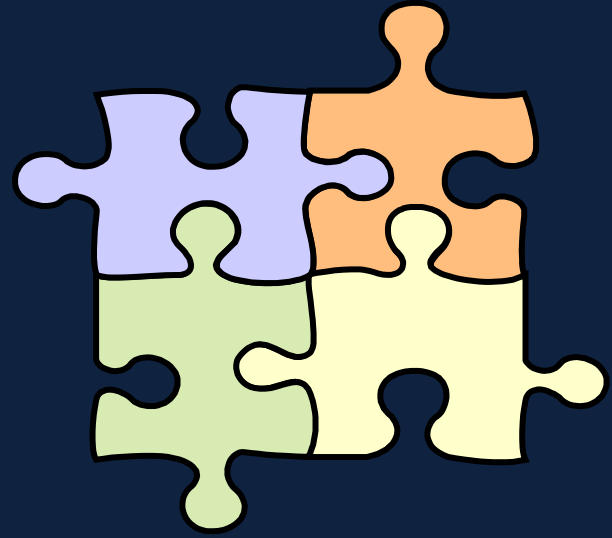
Strengths

- Delivers working software frequently — builds trust with stakeholders
- Continuous feedback reduces the risk of building the wrong thing
- Simple to understand; teams can be trained quickly
- Retrospectives create a culture of continuous improvement
- Works well for complex product development where requirements change

Challenges

- Requires cultural change — not just renaming roles and scheduling meetings
- Product Owner must be available and empowered — often a struggle in practice
- Scrum Master is frequently misunderstood as a project manager
- Does not scale naturally to very large or distributed teams
- Produces less documentation — can be a problem in regulated industries

Extreme Programming



Engineering excellence taken to the extreme — Kent Beck, 1999

What is Extreme Programming (XP)?

Definition

XP is an agile methodology focused on engineering excellence and high-quality software delivered in frequent, short releases. It pushes proven software development practices to their extreme.

XP's philosophy: if something works, do it as much as possible.

✓ Testing works?

→ Write tests BEFORE you write code (TDD)

✓ Code review works?

→ Review code continuously — by pairing (Pair Programming)

✓ Design works?

→ Refactor and redesign constantly — not just at the start

✓ Simple is good?

→ Always do the simplest thing that could possibly work

✓ Short iterations?

→ Deliver new releases to customers every 1–3 weeks

XP — Five Core Values

Communication

Constant face-to-face interaction — the team talks first, documents second

Simplicity

Always do the simplest thing that works. Avoid complexity until it is necessary

Feedback

Fast, frequent feedback from tests, customer, and team — the shorter the loop the better

Courage

Tell the truth about progress, throw away bad code, accept and adapt to late changes

Respect

Every team member's contribution matters — mutual respect enables collective ownership

XP — 12 Core Practices

Three groups: Programming, Team, and Planning & Feedback

Test-Driven Development

Write the test before the code. Red → Green → Refactor. Tests as specification.

Pair Programming

Two developers, one machine. Driver writes; Navigator reviews and plans.

Refactoring

Continuously improve internal code structure without changing external behaviour.

Simple Design

Do only what is needed now. No speculative features. YAGNI (You Ain't Gonna Need It).

Continuous Integration

Integrate and build multiple times per day. Never let integration debt accumulate.

Coding Standards

Shared standards enable collective ownership and refactoring without friction.

Collective Code Ownership

Anyone can improve any part of the codebase at any time.

Sustainable Pace (40-hour week)

No overtime. Tired engineers make mistakes. Sustainable pace is a quality practice.

On-site Customer

Real customer embedded in the team — writes stories, runs acceptance tests, answers questions daily.

Planning Game

Customer defines business value (stories); developers estimate effort. Negotiated release planning.

Small Releases

Release to production frequently — weeks, not months. Real feedback from real users.

Metaphor

Shared system description — common vocabulary and architectural vision understood by all.

XP — Key Engineering Practices

XP is famous for 12 practices — here are the six most important

Test-Driven Development (TDD)

Write the test first. Then write just enough code to make it pass. Then refactor.

Pair Programming

Two developers, one computer. Driver writes code; observer reviews and thinks ahead. Switch regularly.

Refactoring

Continuously improve the internal structure of code without changing its external behaviour.

Continuous Integration

Every developer integrates their code into the shared codebase multiple times per day.

Small Releases

Deliver a working system to customers frequently — in weeks, not months.

Collective Code Ownership

Any team member can improve any part of the code at any time. No individual owns a module.

XP works best in technically strong, collaborative teams. It requires discipline — especially around TDD and pair programming.

XP — Test-Driven Development (TDD)

"Write the test before you write the code"

In TDD, you never write production code unless you have a failing test first. Each small cycle takes just a few minutes.

The TDD cycle — Red → Green → Refactor:

Red

Write a test for a small piece of new behaviour. Run it. It **MUST** fail — if it passes, the test is wrong.



Green

Write the minimum code to make the test pass. No more, no less. Quality does not matter yet — just pass the test.



Refactor

Clean up the code while keeping all tests passing. Now improve the design without changing behaviour.

TDD does not eliminate defects — it catches them early, when fixing them costs almost nothing.

User Stories — Capturing Requirements Simply

XP and Scrum both use user stories as the standard way to express requirements

Structure of a user story

As a [type of user], I want to [do something], so that [I achieve some goal].

Example user story:

Title: User Login

"As a registered user, I want to log in with my email and password, so that I can access my personal dashboard."

A good user story is INVEST:

I Independent: Stories stand alone — not dependent on another story

N Negotiable: A reminder for conversation — not a fixed contract

V Valuable: Delivers value to the end user or customer

E Estimable: Team can estimate the effort to complete it

S Small: Completable within one sprint (usually a few days)

T Testable: Has clear acceptance criteria that can be verified

XP — Strengths and When to Use

Strengths

- High code quality through TDD and constant review
- Low defect rates — bugs caught immediately by tests
- Team flexibility through collective ownership
- Sustainable pace — the 40-hour week principle

Best suited when...

- Small (2–10 person) technically strong team
- Customer available for frequent interaction
- Requirements evolve rapidly
- Engineering quality is the top priority

Scrum vs XP: how do they relate?

Scrum is a management framework — it organises who does what and when. XP is an engineering discipline — it specifies how the code is written. Many teams successfully combine Scrum's sprint structure with XP's technical practices.

Kanban

Visualise work · Limit work-in-progress · Manage flow

What is Kanban?

Definition

Kanban is a method for visualising work, limiting how much work is in progress, and managing the flow of value from request to delivery. Unlike Scrum, it has no fixed sprints, roles, or ceremonies.

Origin of the word:

"Kanban" is Japanese (看板) for sign-board or card. Toyota invented it in the 1940s to signal when to replenish parts on a production line. David Anderson adapted the idea for software development in 2007.

Kanban's three core properties:

Visualise the workflow:

All work is made visible on a Kanban board — nothing is hidden or assumed

Limit Work-In-Progress (WIP):

Constraints on how many items can be in progress at once — drives focus and flow

Manage and improve flow:

Measure and continuously improve how fast work moves from start to done



POWER OF KANBAN EXPLAINED

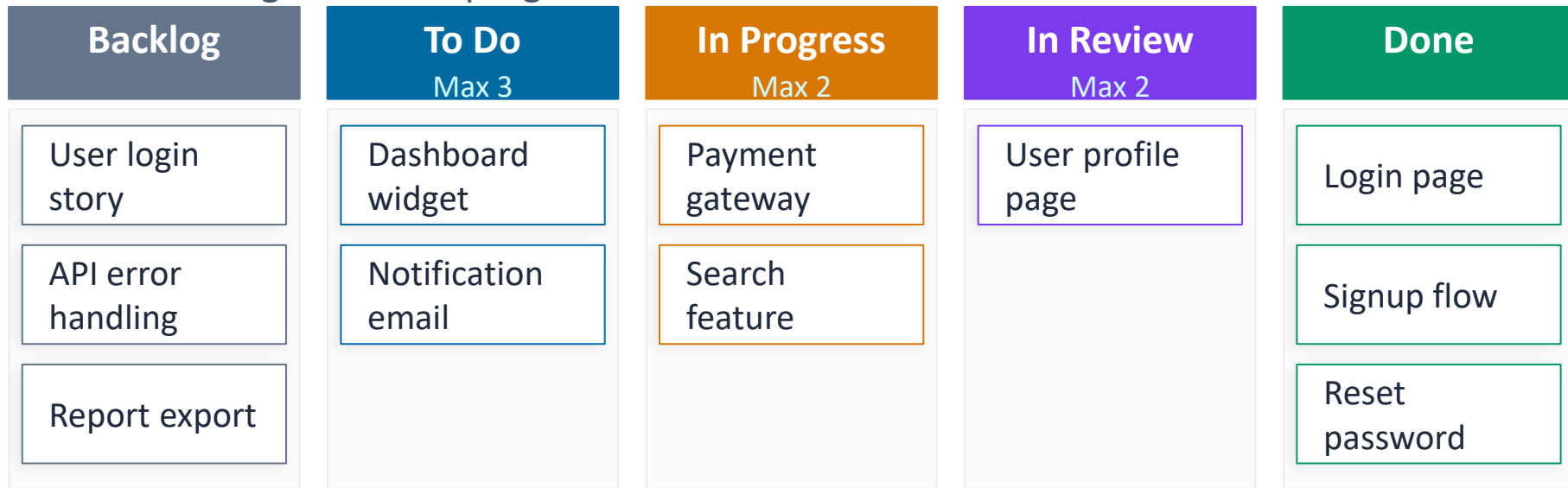
IN LESS THAN 5 MINUTES



The Kanban Board

The board makes all work visible — it is the team's shared reality

A Kanban board has columns representing stages of your workflow. Cards (work items) move left to right as work progresses.



WIP limits prevent the 'In Progress' and 'In Review' columns from growing indefinitely — they force the team to finish before starting new work.

Work-In-Progress (WIP) Limits

The most counterintuitive — and most powerful — Kanban concept

What is a WIP limit?

A WIP limit is a maximum number of work items allowed in a workflow stage at any time. When a column is full, no new work can enter until something exits.

Why limiting work-in-progress improves throughput:

Without WIP limits...

People start many things but finish few. Bugs are discovered late. Context-switching kills productivity. The team looks busy but delivers slowly.

With WIP limits...

Bottlenecks become visible immediately. The team swarms to fix blocked items. Less context-switching. Faster flow from start to done.

"Stop starting. Start finishing." — the golden rule of Kanban

Kanban — Six Core Practices

1

Visualise the workflow

Make all work visible. Map every step from customer request to delivery. Nothing should be hidden in someone's inbox or head.

2

Limit WIP

Set and enforce WIP limits for each stage. When a column is full, stop starting and help finish what is already started.

3

Manage flow

Monitor how work moves through the system. Identify and remove bottlenecks. Track cycle time and throughput.

4

Make policies explicit

Write down the rules for how work moves — what 'ready' and 'done' mean. Visible policies reduce arguments and misunderstandings.

5

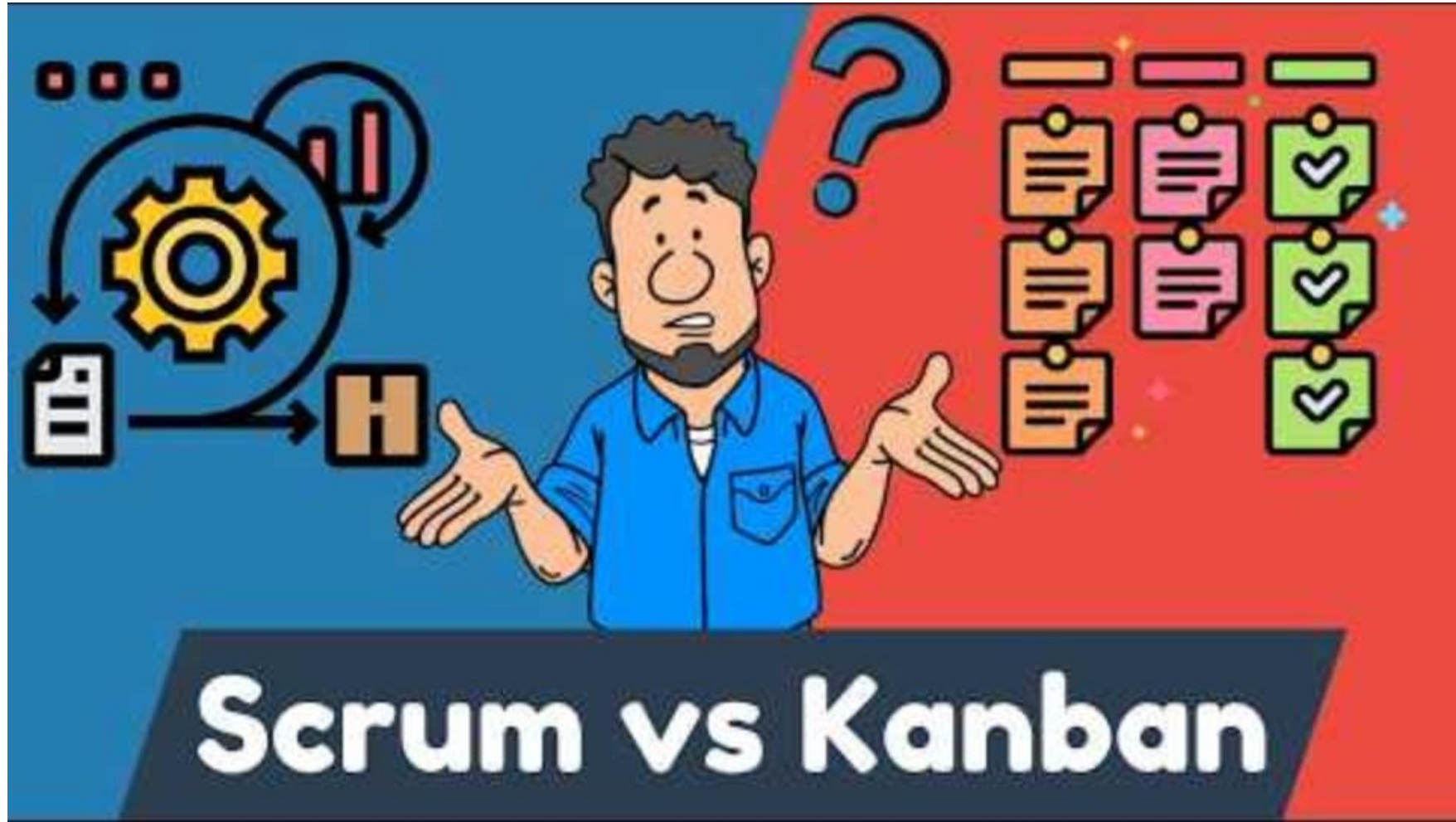
Feedback loops

Hold regular reviews. Daily stand-ups, weekly replenishment meetings, and monthly service delivery reviews improve the system.

6

Improve collaboratively

Use data — cycle time, throughput, cumulative flow — to make evidence-based improvements. Small, evolutionary changes.



Scrum vs Kanban — Key Differences

Both are agile — they solve different problems in different contexts

Factor	Scrum	Kanban
Cadence	Fixed sprints (1–4 weeks)	Continuous flow — no sprints
Roles	Scrum Master, Product Owner, Dev Team	No prescribed roles
Planning	Sprint Planning every sprint	On-demand — replenish when capacity exists
WIP limits	Implicit (sprint capacity)	Explicit — enforced per column
Change	No changes mid-sprint	New items can be added anytime
Output metric	Velocity (story points/sprint)	Cycle time, throughput
Best for	Product development with clear sprints	Continuous service, support, maintenance

Many teams combine both: Scrum for new feature development + Kanban for bug fixes and support — this is called Scrumban.

Lean Software Development

Eliminate waste · Amplify learning · Deliver fast · The intellectual ancestor of Agile

Lean — Origins and Philosophy

Lean did not start in software — it started on the Toyota factory floor

Origins

Toyota Production System (TPS), developed in the 1950s–70s, revolutionised manufacturing by eliminating all forms of waste. Mary and Tom Poppendieck adapted these principles for software in their 2003 book Lean Software Development.

The central idea of Lean:

Identify and deliver value to the customer. Identify every step in the value stream. Eliminate every step that does NOT add value. Then make the value-adding steps flow without interruption.

The Lean connection to Agile:

- The Agile Manifesto was directly influenced by Lean thinking
- Kanban is essentially a Lean technique applied to software workflow
- DevOps adopted Lean's value stream mapping as a core practice
- Many agile principles — simplicity, flow, sustainable pace — come directly from Lean

<https://youtu.be/q5rCzxl77aY>



LEAN SOFTWARE DEVELOPMENT

Lean Software Development — Seven Principles

- | | | |
|---|------------------------------------|---|
| 1 | Eliminate Waste | Remove anything that does not add value — partially done work, extra features, waiting, unnecessary processes |
| 2 | Amplify Learning | Use short feedback cycles and experimentation to accumulate knowledge quickly |
| 3 | Decide as Late as Possible | Delay irreversible decisions until the last responsible moment — you will have more information |
| 4 | Deliver as Fast as Possible | Short cycle times reduce waste, increase feedback, and lower business risk |
| 5 | Empower the Team | Decisions are made by those doing the work — the team closest to the problem has the best knowledge |
| 6 | Build Integrity In | Quality is not tested in at the end — it is built in from the start through good design |
| 7 | See the Whole | Optimise the entire system, not individual parts — local optimisation often creates global waste |

Lean — Seven Types of Waste in Software

Any activity that consumes time or resources without delivering customer value

- | | |
|-------------------------------|--|
| 1. Partially done work | <i>Code that is 90% complete and sitting in a branch for weeks</i> |
| 2. Extra features | <i>Building features nobody asked for — "it might be useful someday"</i> |
| 3. Re-learning | <i>Solving the same problem again because knowledge was not documented</i> |
| 4. Handoffs | <i>Passing work between teams introduces delays and miscommunication</i> |
| 5. Task switching | <i>Moving between multiple work items at once — all progress slows</i> |
| 6. Delays and waiting | <i>Waiting for approvals, environments, test results, or another team</i> |
| 7. Defects | <i>Bugs — especially those found late when the cost of fixing is highest</i> |

Other Agile Methods — Crystal and FDD

At awareness level — knowing these exist and when they are relevant

Crystal

Developed by Alistair Cockburn. A family of methods that vary based on team size and project criticality.

- Crystal Clear (1–6 people) · Crystal Yellow (7–20) · Crystal Orange (21–40) · Crystal Red (41–80)
- All share a focus on people, interaction, community, skills, and communication — not heavy process
- When to use: small to medium teams wanting a lightweight, people-centric approach with flexibility on practices

Feature-Driven Development (FDD)

Developed by Jeff De Luca and Peter Coad. Work is planned and tracked by individual features.

- Five processes: Develop an overall model · Build feature list · Plan by feature · Design by feature · Build by feature
- Features are defined as: 'action + result + object' — e.g. 'Calculate the total of a sale'
- When to use: larger teams (up to 250 people) in enterprise settings where feature tracking matters

3.6

Choosing the Right Agile Approach

One size does not fit all — context determines the best fit

Agile Methods — Comparison at a Glance

Method	Best for	Key strength	Key challenge	Team size
Scrum	Product development	Structure + feedback loops	Role confusion; culture change	3–9
XP	High-quality engineering	TDD, pairing, CI quality	Requires strong tech discipline	2–10
Kanban	Service + maintenance	Flow + WIP limits	No cadence = less predictability	Any
Lean	Waste reduction mindset	Whole-system thinking	Needs experienced practitioners	Any
Crystal	People-focused projects	Flexible and lightweight	Less guidance for new teams	1–80
FDD	Large enterprise	Feature-level tracking	More upfront planning needed	Up to 250

How to Choose — A Simple Decision Guide

Ask these questions about your project context:

Q: Are requirements unclear or evolving rapidly?

→ Scrum or XP (frequent delivery + customer feedback)

Q: Do you need continuous delivery with no fixed cadence?

→ Kanban (manage flow, no sprints required)

Q: Is engineering quality the absolute top priority?

→ XP (TDD, pair programming, continuous integration)

Q: Do you want to eliminate waste across the whole process?

→ Lean thinking + Kanban

Q: Small team, want minimal process overhead?

→ Crystal Clear (very lightweight)

Q: Large enterprise, need to track 100s of features?

→ FDD or Scrum at Scale (SAFe, LeSS)

In practice, most teams mix approaches. Scrum + XP engineering practices is extremely common. Kanban is often added for support work alongside Scrum for new features.

Agile at Scale

What happens when hundreds of people work on the same product?

The challenge

Scrum and Kanban work brilliantly for teams of 3–15 people. Large enterprises with 500+ developers building one product need additional frameworks to coordinate across multiple teams

SAFe

Scaled Agile Framework

The most widely used scaling framework. Organises teams into Agile Release Trains (ARTs). Five levels from team to portfolio.

Use when: Large enterprises with many teams and complex governance requirements

LeSS

Large-Scale Scrum

Scrum applied across multiple teams. Minimises process overhead. One Product Backlog, one Product Owner, many Scrum teams.

Use when: 2–8 teams working on the same product with a preference for lightweight coordination

Nexus

The Nexus Framework

Created by Scrum.org. Extends Scrum for 3–9 teams with a Nexus Integration Team to manage dependencies.

Use when: Medium-scale (30–80 developers) on a single product with strong integration challenges

3.7

Agile Tools

The software that supports agile teams every day

Agile Tools — Why They Matter

A team practicing Scrum or Kanban needs tools to make work visible and coordination easy

What agile tools do

Agile tools replace the physical wall of sticky notes with a digital board that everyone can see, whether they are in the same room or working remotely across the world.

Four types of tools agile teams use:

Project and backlog management

Jira, Azure DevOps, Linear, Monday.com

Visual task boards

Trello, Miro, Asana, ClickUp

Collaboration and communication

Confluence, Notion, Slack, Teams

Reporting and metrics

Jira Burndown, GitHub Insights, Velocity Charts

Jira — The Most Widely Used Agile Project Tool

Developed by Atlassian · Used by over 65,000 organisations worldwide

Jira supports both Scrum and Kanban workflows out of the box.

Product Backlog

Create and manage user stories, bugs, and tasks. Assign story points, labels, and priority. The Product Owner prioritises the backlog in one view.

Sprint Board

The Scrum board shows all items in the current sprint: To Do, In Progress, Done. Team members drag cards as work progresses.

Kanban Board

Configure columns to match your workflow. Set WIP limits per column. Jira enforces them automatically.

Burndown Charts

Auto-generated charts showing remaining work vs time. Instantly visible to the whole team and stakeholders.

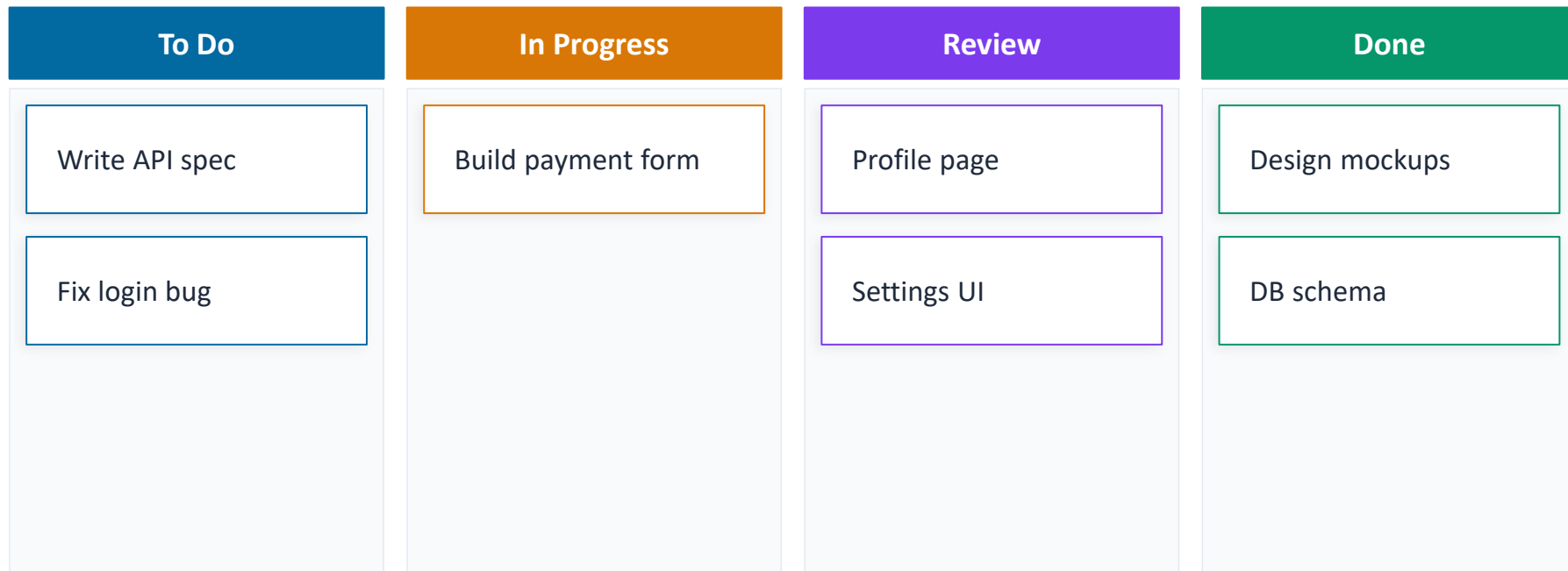
Velocity Reports

Track how many story points the team completes each sprint. Helps forecast future sprint capacity accurately.

Trello — The Simple Visual Board

Atlassian · Ideal for teams starting out with Kanban · Free tier available

Trello is based on Kanban boards. Work is organised into boards → lists → cards.



Trello is ideal for learning. Jira is where most professional teams live. Try Trello first to understand Kanban; move to Jira for Scrum projects.

Other Key Agile Tools to Know

Azure DevOps

by Microsoft

Full DevOps pipeline — boards, repos, pipelines, test plans. Deep integration with Microsoft ecosystem.

Linear

by Linear Inc.

Modern, fast issue tracker popular with software startups. Clean UI, keyboard-first design.

Monday.com

by Monday

Flexible work management platform. Popular outside pure software teams — also used in marketing, HR.

GitHub Projects

by GitHub

Integrated task board directly in GitHub. Ideal for open-source projects and teams already using GitHub.

Miro

by Miro

Virtual whiteboard for retrospectives, sprint planning poker, roadmapping, and team brainstorming.

3.8

AI in Agile Software Development

The next frontier — how AI is changing the way agile teams work

AI is Changing How Agile Teams Work

Not replacing the process — changing what humans focus on within it

The shift

AI tools are being integrated into every stage of the agile cycle — from sprint planning to code review to retrospectives. The engineer's role is shifting from writing every line to directing, reviewing, and validating AI-generated work.

AI does not replace agile teams. It changes what they spend time on:

<i>Before: Writing boilerplate code</i>	→ After: Reviewing, testing, and improving AI-generated code
<i>Before: Manual test case creation</i>	→ After: Verifying and refining AI-generated test suites
<i>Before: Estimating story effort alone</i>	→ After: Using AI predictions as a starting point for discussion
<i>Before: Writing meeting summaries</i>	→ After: Editing AI-drafted summaries of stand-ups and retros

AI in Sprint Planning and Estimation

One of the most time-consuming agile activities — AI can help

Sprint planning typically takes 4–8 hours. Teams debate estimates, discuss stories, and decide scope. AI can assist in several ways:

Story refinement

AI reads raw requirements and suggests user story breakdowns, acceptance criteria, and edge cases the team may not have considered.

Effort estimation

AI models trained on historical project data can predict story point ranges based on similar past work — giving the team a data-informed starting point.

Dependency detection

AI scans the backlog and flags stories that depend on other stories or external systems — preventing blocked sprints before they start.

Risk identification

AI analyses technical complexity and historical data to flag which stories have the highest risk of overrun or defects.

Capacity suggestions

AI incorporates team velocity, planned absences, and historical patterns to suggest a realistic sprint capacity.

AI in Agile Development — Coding and Review

GitHub Copilot

Suggests complete functions, classes, and tests as you type. Trained on billions of lines of open-source code. Works inside VS Code, JetBrains, and other IDEs.

ChatGPT / Claude

Explain unfamiliar code, suggest refactoring, generate unit tests, write documentation, and debug error messages in natural language.

Cursor

An AI-first IDE. You describe what you want to build, and Cursor writes the code, applies changes, and explains what it did.

Tabnine

AI code completion trained on your team's private codebase. Learns your team's patterns, naming conventions, and coding style.

AI-generated code still needs to be read, understood, tested, and owned by the engineer. Professional responsibility does not transfer to the AI.

AI in Testing and Retrospectives

AI in Testing

- Automated test generation: AI generates unit tests from code or acceptance criteria — significantly reducing the manual effort of writing test cases
- Test case prioritisation: AI predicts which tests are most likely to catch regressions based on what code changed — saving time in CI pipelines
- Bug triage and prediction: AI analyses incoming bug reports, classifies severity, and predicts which developer or component is most likely responsible

AI in Sprint Retrospectives

- Meeting transcription: AI transcribes and summarises the retrospective in real time — freeing the team to talk without taking notes
- Sentiment analysis: AI analyses retrospective feedback to identify patterns — what the team consistently struggles with across sprints
- Action item tracking: AI extracts action items from the retro discussion and automatically creates Jira tickets or reminders

AI in retrospectives is most powerful when it shows patterns across many sprints — patterns humans may

AI + Agile — Looking Ahead

This is the frontier — and understanding it now gives you an advantage

The software engineering landscape in 2030 will look very different from today. These are emerging trends you should watch:

AI Product Owners

AI systems that analyse user behaviour, A/B test results, and business metrics to suggest which stories to prioritise — acting as a data-driven input to the Product Owner.

Autonomous code generation

Multi-agent AI systems that plan, build, test, and deploy features from a natural-language description — with humans reviewing and approving, not writing every line.

Predictive sprint health

AI models that predict mid-sprint whether the team will meet the sprint goal — and suggest which items to descope before it is too late to recover.

AI-native development tools

IDEs and platforms where the AI is the primary author and the human is the reviewer and approver — a complete inversion of today's workflow.

The engineers who will thrive are those who understand agile deeply enough to direct, review, and govern AI-generated work — not just write code themselves.

Section 3 — What We Have Covered

Check your understanding against the learning outcomes

- LO1** The Agile Manifesto contains 4 values and 12 principles — together they define what it means to work in an agile culture.
- LO2** Scrum has 3 roles, 3 artifacts, 5 events, and time-boxed sprints. The Definition of Done ensures quality without ambiguity.
- LO3** XP's key practices include TDD, pair programming, refactoring, continuous integration, and collective code ownership.
- LO4** A Kanban board visualises work in columns. WIP limits prevent overloading the team and expose bottlenecks automatically.
- LO5** Lean originated at Toyota. Its 7 principles — eliminate waste, amplify learning, deliver fast, empower team, etc. — underpin all agile thinking.
- LO6** Choosing the right method depends on team size, requirement stability, engineering priorities, and delivery cadence needs.
- LO7** Jira supports Scrum and Kanban; Trello is ideal for learning Kanban. Azure DevOps, Linear, and Miro are also widely used.
- LO8** AI assists in sprint planning, code generation, testing, and retrospectives. Engineers must direct, review, and own AI-generated work.

References and Further Reading

Main Textbooks

Software Engineering

Ian Sommerville, 10th ed., Addison-Wesley, 2015 — Chapter 3: Agile Software Development

Software Engineering: A Practitioner's Approach

Roger S. Pressman, 9th ed., McGraw-Hill, 2020 — Chapters 4–6

Lean Software Development: An Agile Toolkit

Mary & Tom Poppendieck, Addison-Wesley, 2003 — foundational Lean text

Online Resources

- agilemanifesto.org — the original Agile Manifesto with all 12 principles
- scrumguides.org — official free Scrum Guide (Schwaber & Sutherland, 2020 edition)
- kanban.university — Kanban Method resources and guides
- atlassian.com/agile — Atlassian's free agile learning centre (Jira, Trello, Confluence)
- github.com/features/copilot — GitHub Copilot overview and documentation

Coming Up Next

Section 4:

Requirements Engineering

Requirements elicitation · Use cases · User stories · System modelling · Requirements validation

Tutorial this week:

Introduction to Software Engineering ·
Prof. K. P. Hewagamage